

Telegram Rumble BOT 完全ガイド（初心者向け・印刷版）

作成日: 2026-05-25

1. この資料について

この資料は、今回あなたと一緒に作った Telegram Rumble BOT について、初心者でもわかるように「実際にやったこと」「コードの意味」「使った技術の読み方」を全部まとめたものです。

2. 最終的にできたこと（完成機能）

2-1. バトル機能

- /battle で募集開始（管理者限定）
- ボタンで参加
- 募集時間 30 / 60 / 90 秒
- 重複参加防止
- BOTアカウント参加不可
- 勝者決定
- 勝者のみメンション（通知スパム対策）

2-2. 多言語機能

- 日本語 / 英語 / 中国語（簡体字）対応
- ユーザーのTelegram言語設定で自動切替
- /setlanguage auto|ja|en|zh で固定も可能

2-3. Premium機能

- /schedule（自動開催）
- /unschedule
- /setbutton
- /setwinner
- /seteffect
- /setgif

2-4. Stripe決済機能

- /buy で決済リンク作成
- Stripe Webhookで決済完了イベントを受信
- Premium自動有効化
- 決済完了はDM通知のみ（プライバシー対応）

2-5. DM運用機能

- /helpdm で手順表示
- /start（DM）でも手順表示
- /buy -100xxxxxxxxxxx で対象グループ指定購入

3. これまで実際に行った作業（時系列）

1. BOTプロジェクトを新規作成
2. Node.js + Telegraf + SQLite構成を実装
3. 多言語ロケールファイルを作成
4. /battle, /settings, /license など基本コマンド実装
5. Railwayデプロイ設定を追加
6. Stripe本番リンクに差し替え

7. /buy + Webhook 自動認証を実装
8. DMで購入・認証確認できるよう改善
9. /help とコマンド候補（Telegramメニュー）を実装
10. グループ内 help/settings の自動削除（2分）を追加
11. Premium復旧用オーナーコマンド /premiumon を追加
12. DATABASE_URL（PostgreSQL）対応を追加

4. なぜトラブルが起きたか

4-1. 409 Conflict エラー

意味: 同じBOTトークンで複数起動され、TelegramのgetUpdatesが競合。

対策:

- 不要な起動元を停止
- BOTトークン再発行
- Railwayのみ稼働に統一

4-2. PremiumがOFFに戻る

意味: 保存先問題（再起動で状態が消える/同期されない）。

対策:

- PostgreSQL（DATABASE_URL）側へPremium状態を保存
- 必要時は /premiumon で復旧

5. 現在の正しい購入フロー

グループ購入

1. 管理者が /buy
2. 決済リンク発行
3. Stripeで支払い
4. Webhook受信
5. Premium有効化
6. DMに完了通知

DM購入

1. グループで /groupid
2. DMで /buy -100xxxxxxxxxx
3. Stripeで支払い
4. Webhook受信
5. Premium有効化
6. DMに完了通知

6. 主要コマンド一覧

グループ

- /help
- /battle
- /settings

- /groupid
- /setduration 30|60|90
- /setlanguage auto|ja|en|zh
- /buy
- /premium
- /schedule 21:00
- /unschedule
- /setbutton <text>
- /setwinner <template>
- /seteffect add <text>
- /seteffect reset
- /setgif on|off
- /license XXXXX-XXXXX
- /premiumon (オーナー)

DM

- /start
- /help
- /helpdm
- /buy -100xxxxxxxxxx
- /premium
- /license XXXXX-XXXXX -100xxxxxxxxxx
- /premiumon -100xxxxxxxxxx (オーナー)

7. コードの解説 (初心者向け)

src/index.js

BOTの本体です。

- コマンド受付
- 管理者チェック
- バトル進行
- Stripe決済連携
- Webhook受信
- Premium有効化
- DMガイド表示

src/db.js

SQLiteのテーブル定義です。

主なテーブル:

- groups
- settings
- licenses
- battles
- players
- schedules
- stripe_events

src/services/i18n.js

多言語切替の中核です。

- 言語判定
- 翻訳文取得
- 演出文取得

src/services/utlis.js

時刻やスケジュール引数の補助処理です。

src/locales/ja.json / en.json / zh.json

各言語の表示文を管理します。

8. 使用した言語・技術（読み方つき）

- Node.js（ノードジェイエス）
 - JavaScriptをサーバーで動かす実行環境
- JavaScript（ジャバスクリプト）
 - BOTロジックを書く言語
- Telegraf（テレグラフ）
 - Telegram BOT開発ライブラリ
- SQLite（エスキューライト）
 - 軽量DB
- PostgreSQL（ポストグレスキューエル）
 - 本格DB（永続化に強い）
- Stripe（ストライプ）
 - 決済サービス
- Webhook（ウェブフック）
 - 外部イベント通知の仕組み
- JSON（ジェイソン）
 - 設定/翻訳データ形式
- dotenv（ドットエンブ）
 - .envから環境変数を読む仕組み
- Railway（レイルウェイ）
 - デプロイ先クラウド
- Git / GitHub（ギット / ギットハブ）
 - ソース管理
- Markdown（マークダウン）
 - ドキュメント記法

9. Railwayに必要な主な環境変数

- BOT_TOKEN
- DATABASE_PATH
- DATABASE_URL
- STRIPE_PAYMENT_LINK

- STRIPE_SECRET_KEY
- STRIPE_PRICE_ID
- STRIPE_WEBHOOK_SECRET
- STRIPE_SUCCESS_URL
- STRIPE_CANCEL_URL
- LICENSE_SECRET
- PREMIUM_LICENSE_SEEDS
- BOT_OWNER_ID
- DEFAULT_TIMEZONE

10. Stripe側の必須設定

1. 本番モード
2. sk_live キー取得
3. 価格 (price_...) 作成
4. Webhook追加
 - URL:
https://<railway-domain>/stripe/webhook
 - Event: checkout.session.completed
5. whsec をRailwayへ設定

11. 運用チェックリスト

毎日:

- RailwayがActiveか
- Deploy Logsにエラーがないか

決済後:

- DM完了通知が届くか
- /settings で Premium: ON か

障害時:

- 409が出ていないか
- 必要なら /premiumon で復旧

12. まとめ

今回のBOTは、無料バトル機能 + Premium決済機能 + 多言語対応 + DM運用を備えた実運用向け構成になっています。

特に重要なのは次の3点です。

1. 決済フローを自動化したこと
2. 通知スパムやプライバシーに配慮したこと
3. トラブル時に復旧できる運用コマンドを用意したこと

これで、今後の拡張（ランキング、XP、TON/USDT、NFT機能）にも進みやすい土台が整っています。